

Remix: Postman to Pytest Migrator

Migrate Postman Collection JSON requests into clean, idiomatic Python pytest functions using Gemini AI.

[Project Documentation — 6 Core Documents](#)

[Team Members](#)

N R Akash

R Harshavarthini

G S Kowshika

Why these documents matter

AI coding agents are powerful — but they're not mind readers. The more context you give them upfront, the better and faster they build.

These 6 documents eliminate ambiguity. Instead of the AI guessing what stack to use, how screens connect, or where data lives — it knows. The result: fewer hallucinations, less back-and-forth, cleaner code.

Think of these as the briefing you'd give a senior developer on Day 1. The more complete the brief, the better the output.

#	Document	One-line purpose
01	PRD — Product Requirements Doc	What you're building and for whom
02	TRD — Technical Requirements Doc	Which tech, tools, and APIs you'll use
03	App Flow	Every page, every click, every navigation path
04	UI/UX Design Brief	How your app looks and feels
05	Backend Schema	Data structure, tables, auth, relationships
06	Implementation Plan	The exact step-by-step build sequence

01

PRD — Product Requirements Document

The north star. Everything else flows from this.

Why it matters

The PRD is the first document you write — before any code, design, or architecture. It defines what you're building, who it's for, and what success looks like. Without a PRD, your AI agent fills in the blanks with assumptions. A solid PRD means fewer surprises.

What to include

- App name and tagline
- The problem you're solving and for who
- Core value proposition (what makes your app different)
- Target user persona — describe them in 2–3 sentences
- List of all features (split into Must Have / Nice to Have)
- User stories — 'As a [user], I want to [do X] so that [Y]'
- Out of scope — explicitly list what this version does NOT do
- Success metrics — how you'll know the product is working

Document 01 — PRD

App Name	Remix: Postman to Pytest Migrator
Tagline	Migrate Postman Collection JSON requests and their JavaScript test assertions into clean, idiomatic Python pytest functions using Gemini AI.
Problem	Translating graphical API collections and proprietary JavaScript assertions (pm.* / Chai styles) into Python-based enterprise quality suites involves heavy manual work. Dev and QA teams want to avoid vendor lock-in, utilize Python-born testing ecosystems, and integrate standalone scripts into CI/CD pipelines without rewriting thousands of assertions by hand.
Target User	QA automation engineers, SDETs, and backend developers looking to export standard, PEP8-compliant integration and dynamic regression test suites in Python.
Core Features (Must Have)	1. Dual-Channel Collection Import: Drag & drop Postman JSON (V2/V2.1) OR fetch via Postman API Key 2. Granular Request Inspector: split-screen explorer for params, headers, body, test scripts 3. Generative Migration Engine: server-side conversion via Gemini AI 4. Multi-File Outputs: consolidated test_all_apis.py + modular scripts 5. Interactive Simulation CLI: sandbox for success, timeouts, 401s, schema mismatches 6. AI-Driven Failure Diagnosis: root-cause explanations + copy-pasteable patches 7. Dynamic ZIP Packaging: bundles entire migrated suite

Nice to Have	Interactive terminal execution checking for local native Python installations. Custom developer prompt injection for custom Python syntax overrides.
Out of Scope	Executing arbitrary, untrusted Python on the hosting platform's live server (all runs are safely simulated). Direct translation of proprietary web-browser libraries.
User Stories	As a QA Engineer, I want to upload a Postman JSON file so that I can automatically convert it to robust Python code without manual translation. As an SDET, I want to fetch collections live via the Postman API so that I don't need to manually export and import files repetitively. As a Lead Developer, I want to simulate test execution with schema drift reports so that I can identify code gaps and adapt tests immediately using recommended patches.
Success Metrics	100% successful conversion of baseline Postman assertion commands (pm.response.to.have.status(), pm.expect()). Average workspace loading latency below 100ms. Migration transaction return time below 2.5s utilizing Gemini flash optimization.

02

TRD — Technical Requirements Document

The blueprint your AI agent needs to make technical decisions without guessing.

Why it matters

The TRD translates your product vision into technical decisions. It answers: what language, what framework, what database, what third-party services. When you skip this, your AI picks a stack on its own — sometimes well, often inconsistently. The TRD locks in the choices so the agent stays coherent across every file it writes.

What to include

- Frontend framework (e.g. React, Next.js, Vue)
- Backend framework or runtime (e.g. Node/Express, Python/FastAPI)
- Database type and provider
- Authentication method
- Hosting and deployment
- Third-party APIs and services
- Folder structure and naming conventions
- Environment variables needed
- Any hard technical constraints or preferences

Document 02 — TRD

Frontend	SPA built on React 19, TypeScript, and Vite 6 styled with Tailwind CSS v4. Interface animations powered by motion (motion/react). Icons: lucide-react.
Backend	Node/Express backend serving API endpoints and loading Vite middleware in development. Bundled with esbuild into dist/server.cjs in production. Runs on Cloud Run (Port 3000).
Database	Entirely stateless. Sensitive secrets and Postman details exist only in active memory or localStorage buffers. No SQL or server databases required.
Auth	Secure input forms on the client-side proxying credentials (Postman Keys, Gemini Keys) safely through server-side controller APIs.
Hosting	Google Cloud Run infrastructure.
Third-party APIs	Google Gemini API (@google/genai SDK, gemini-3.5-flash with fallback to gemini-3.1-flash-lite / gemini-flash-latest). Postman API (api.getpostman.com): /workspaces, /collections, collection details via user tokens.

Key Libraries	react, react-dom (v19) express (v4/v5) @google/genai (v2.4+) adm-zip (in-memory ZIP builder) motion (animations) lucide-react (icons)
Environment Variables	GEMINI_API_KEY — server secret for Gemini AI engine. APP_URL — automated instance routing path from Cloud Run.
Constraints	Large file payload processing (JSON body up to 50MB). Strict sandbox network confinement bound to Port 3000.

03

App Flow — Navigation & User Journey Map

Every page, every click, every path — mapped before a single screen is built.

Why it matters

The app flow tells your AI exactly how users move through your product. Without this, the AI builds pages in isolation. The app flow is what makes your product feel like one coherent thing.

Document 03 — App Flow

Pages List	Single-page application (SPA). Left Navigation Rail with three dynamic menus: Explorer (Workspace Editor), Settings (API Keys & Testing Presets), Documentation (Quickstart Guides).
Navigation Type	Responsive left navigation rail + dynamic workspace main panel with double-paned layout. Sub-tabs: Consolidated Code, Modular Modules, Interactive Sandbox CLI, Reference Manual.
First Screen	Interactive Workspace with file drop zones, Postman Live-Connect inputs, and instant-load examples (User Management Service, E-Commerce Checkout Flow).
Auth Flow	Zero login flow. Configuration settings loaded on-the-fly and preserved locally via localStorage browser states.
Core User Journey 1	1. User lands on dashboard, selects a built-in model or drags in Postman collection 2. Postman hierarchy loads in center-left tree; user clicks endpoints to verify 3. User adjusts compilation parameters in Settings (library, host URLs, mapping comments) 4. Clicks 'Migrate to Pytest' — triggers Gemini prompt 5. Console loads fully rendered Python script with highlights or modular file tabs 6. User clicks 'Download Suite (ZIP)' to retrieve finished workspace code
Core User Journey 2	1. After migrating, user opens Sandbox Run tab 2. Selects simulation state (Success / Connection Error / Auth Drift / Schema Mismatch) 3. Clicks 'Execute Suite' — spins up diagnosis loop 4. Colored Python log outputs in real-time with failure summaries and tracebacks 5. AI analysis card presents failure cause, recommendations, and copyable code patch
Empty States	Dashboard panels with interactive icons (UploadCloud, Sliders) explaining what configuration files are missing.
Error States	Red inline cards detailing connectivity issues, invalid structure files, or missing Gemini secrets (triggers a 449 setup callback).
Redirects	No heavy external transitions. Responsive React page-renders avoid jarring refreshes.

04

UI/UX Design Brief — Visual & Interaction Design Guide

So the AI doesn't just build something functional — it builds something you'd actually use.

Why it matters

AI agents default to plain, unstyled, or inconsistent UIs when left to their own judgment. The design brief locks in your visual language: colors, fonts, component style, spacing, and overall vibe. Even a rough direction gives the AI enough to make coherent decisions across every screen.

Document 04 — UI/UX Design Brief

Aesthetic	Visual layout matching professional development tools like VS Code, Raycast, and Linear. Clean, high-contrast, developer-friendly interface panels.
Primary Background	Dark: #0b0f19 (midnight) with #0f172a workspace cards. Light mode: pristine light gray desktop.
Accent / CTA Color	#6366f1 — deep tech purple/violet for main user interactions and system transitions.
Typography	Headers & Titles: Sora and Plus Jakarta Sans. Monospace: JetBrains Mono for API URLs, JSON blocks, terminal code displays, and tab paths.
Border Radius	8px and 12px rounded corners throughout. Crisp, low-weight borders (#1e293b).
Shadows	Subtle card shadows. No clutter or excessive status details in margins.
Dark/Light Mode	Full light-dark toggle on workspace header. Dark defaults to Midnight interface; Light converts to paper-white with charcoal text.
Reference Apps	VS Code, Raycast, Linear.
Mobile	Full page columns collapse into overlay drawers on tablet views. Large touch targets (44px+) for control buttons.

05

Backend Schema — Data Model & Auth Architecture

How your data is stored, structured, and secured.

Why it matters

The backend schema is the hardest document to change after the fact. Once your database has data in it, restructuring tables is painful. Getting this right upfront saves hours of debugging and migration hell. Your AI agent uses this to write database migrations, API routes, and auth logic.

Document 05 — Backend Schema

Storage Architecture	This application does not write raw inputs or generated credentials to any database. 100% of data processing occurs on-the-fly in security-isolated memory spaces.
Browser Storage	pytestify_theme: 'dark' 'light' (localStorage)
GET /api/examples	Distributes pre-formatted Postman schemas for quick-play.
POST /api/postman/workspaces	Tests developer keys and queries cloud workspace groups.
POST /api/postman/collections	Lists available target collections inside a given workspace.
POST /api/postman/fetch	Downloads full targets and extracts flattened, unified request arrays.
POST /api/migrate	Central AI migration engine. Maps HTTP structures and JavaScript test scopes using gemini-3.5-flash (returns detailed assertions and modular Python structures).
POST /api/execute	Sandbox CLI execution engine. Generates realistic terminal outcomes for selected scenario, returning analytical reviews and code patches.
POST /api/download-zip	Dynamic ZIP compiler using adm-zip to stream in-memory files.
Auth Provider	Secure client-side input forms proxying credentials through server-side APIs. No JWT, no OAuth, no user accounts required.
Sensitive Fields	Postman API keys and Gemini API keys are never persisted to disk or database. Processed in-memory only.

06

Implementation Plan — Step-by-Step Build Sequence

The exact order to build so the AI never skips a foundation layer.

Why it matters

The implementation plan is your build roadmap. It tells the AI agent what to build first, second, and third — and why that order matters. Skipping this leads to the AI building the frontend before the backend exists, or adding auth after the DB schema is half-baked. Clear phases, clear milestones, clear completion criteria.

Document 06 — Implementation Plan

Phase 1: Setup	Verify directory structure and compile tools. Install core production libraries (@google/genai, adm-zip, motion, tailwind). Set up GEMINI_API_KEY and APP_URL environment variables.
Phase 2: Postman Controller API	Implement Express-side data router handlers: /api/examples, /api/postman/workspaces, /api/postman/collections, /api/postman/fetch.
Phase 3: Gemini Migration Engine	Build POST /api/migrate — map HTTP structures, auth headers, body types, dynamic environment states, and JavaScript Chai test structures to Python pytest syntax via gemini-3.5-flash.
Phase 4: Sandbox Execution Engine	Build POST /api/execute — simulate terminal outcomes (Success, Connection Error, Auth Drift, Schema Mismatch). Return AI analysis cards with root-cause explanations and code patches.
Phase 5: ZIP Packaging	Implement POST /api/download-zip using adm-zip to bundle consolidated test_all_apis.py + modular test files (test_auth.py, test_products.py, etc.) into a single .zip.
Phase 6: Frontend SPA	Build React 19 + Vite 6 + Tailwind CSS v4 SPA. Implement left navigation rail, split-screen explorer, file drop zones, Postman Live-Connect input, and code output tabs.
Phase 7: UI Polish	Implement responsive design, overlay drawers on mobile, loading states, empty states (UploadCloud icons), error states (red inline cards), dark/light mode toggle.
Phase 8: Testing	Manual test all flows: file import, live API fetch, migration, simulation, ZIP download. Fix edge cases. Validate all baseline pm.* assertion conversions.
Phase 9: Deploy	Configure Google Cloud Run. Set production environment variables. Validate Port 3000 binding, 50MB payload limit, sandbox network confinement.
Done Criteria	All core flows work end-to-end: import → inspect → migrate → simulate → diagnose → download. Migration latency under 2.5s. 100% conversion of baseline Postman assertions.

How to use these documents

You don't have to write all 6 from scratch manually. Use Claude or ChatGPT to help you fill them out — just paste the template, answer the prompts conversationally, and let the AI clean it up into a proper document.

Suggested workflow

1. Start with the PRD — get your product idea out of your head and into writing
2. Fill in the TRD — decide your stack before anything else
3. Map your App Flow — draw it out, then describe it in text
4. Write the UI/UX brief — even just 'dark mode, minimal, like Notion' helps
5. Design your Backend Schema — tables first, relationships second
6. Write your Implementation Plan — phases, not just a flat task list

Once all 6 are done, paste them at the start of your AI agent conversation. Tell it: 'Here are my project documents. Use these as the source of truth for everything you build.'

The 10 minutes you spend on these documents saves hours of fixing broken code, re-explaining context, and undoing bad architectural decisions.